

CHECKLIST — O QUE PRECISA SER FEITO PARA MELHORAR O RADAR FX

PRIORIDADE ALTA (🔥 Impacto direto na estabilidade)

Infraestrutura & Confiabilidade

- [] Criar **Windows Service** para o servidor Node.js (iniciar automaticamente com o Windows)
- [] Criar **Windows Service** para a Bridge Python (iniciar automaticamente com o Windows)
- [] **Health check endpoint** com métricas detalhadas (uptime, memória, status de cada engine)
- [] **Rate limiting** nas APIs públicas (evitar sobrecarga acidental)
- [] **Graceful shutdown** — salvar estado de todos os engines ao desligar
- [] **Log rotation** — logs atuais crescem sem limite; implementar rotação diária
- [] **Armazenamento de senha do Telegram** — atualmente em plaintext JSON; mover para env var ou cofre

Bridge MT5

- [] **Reconexão automática** — se a Bridge MT5 cair, tentar reconectar em vez de só logar erro
- [] **Fila de ordens** — implementar fila para evitar perda de ordens se a bridge estiver ocupada
- [] **Timeout configurável** — aumentar timeout em operações de ordem (algumas levam >5s)
- [] **Health check entre servidor e bridge** — monitoramento bidirecional

Persistência & Estado

- [] **Salvar estado automaticamente a cada X minutos** (não só no update) — evitar perda em crash
- [] **Backup automático dos arquivos JSON** — copiar settings para pasta `backups/` diariamente
- [] **Verificar engines que faltam settings file** — alguns motores podem não ter persistência

PRIORIDADE MÉDIA (⚡ Performance & Precisão)

Conflitos entre Robôs

- [] **Evitar que dois robôs operem o mesmo símbolo simultaneamente** — Alpha Robot e Gold Scalper podem ambos operar XAUUSD
- [] **Sistema de lock por símbolo** — implementar semáforo para evitar ordens concorrentes
- [] **Orquestrador de robôs** — coordenar qual robô opera quando, baseado em prioridade

Otimização de Memória

- [] **Limpar alertas antigos** — AlertEngine mantém todos os alertas em memória; limpar >24h
- [] **Limpar histórico de trades na memória** — engines mantêm arrays crescentes; limitar a 1000
- [] **Limpar `signal_tracker.json`** — arquivo cresce indefinidamente; podar entradas antigas
- [] **Garbage collection** — forçar GC em momentos de baixa atividade

Banco de Dados

- [] **Implementar Prisma** (já está nas dependências mas não é usado) — substituir JSON files
- [] **Migrar settings para banco** — mais seguro que JSON files para escrita concorrente
- [] **Migrar trade history para banco** — consultas mais eficientes que JSON

Tratamento de Erros

- [] **Substituir catch silenciosos** — muitos `catch { }` sem log; dificulta debug
- [] **Circuito de retry com backoff** — em vez de falhar imediatamente, tentar 3x com intervalo
- [] **Timeout global para chamadas à bridge** — evitar que uma chamada trave o engine
- [] **Notificação de erros críticos** — enviar alerta Telegram quando um engine falhar

PRIORIDADE BAIXA (🛠️ Melhorias & Features)

Testes

- [] **Testes unitários** para o SignalEngine (lógica de classificação)
- [] **Testes unitários** para o AlertEngine
- [] **Testes de integração** — servidor + bridge mock
- [] **Modo simulação** — executar todos os robôs em modo paper trading

Qualidade de Código

- [] **TypeScript strict mode** — ativar `strict: true` no tsconfig e corrigir erros
- [] **Remover dead code** — arquivos `.bak`, imports não usados, funções comentadas
- [] **Remover Prisma das dependências** se não for usar, ou implementar de vez
- [] **Padronizar nomenclatura** — mistura de camelCase, snake_case nos JSONs
- [] **Adicionar ESLint + Prettier** ao projeto

Frontend

- [] **Code splitting** — carregar componentes pesados sob demanda (lazy loading)
- [] **Bundle analysis** — verificar tamanho do bundle e identificar otimizações
- [] **Cache de respostas da API** — evitar chamadas repetidas para os mesmos endpoints
- [] **Modo escuro consistente** — verificar se todos os componentes seguem o tema
- [] **Responsivo** — garantir que todas as telas funcionem em mobile
- [] **Estado global** — usar Zustand para estado compartilhado (já está nas deps)

Segurança

- ☐ **Autenticação** — adicionar login básico na API (nunca expor para internet sem auth)
- ☐ **Token Telegram em env var** — remover de telegram_settings.json
- ☐ **Sanitização de input** — validar parâmetros recebidos nas APIs
- ☐ **CORS configurado** — atualmente parece aberto para todas origens

Monitoramento

- ☐ **Métricas de performance** — tempo de resposta da bridge, latência de ordens
- ☐ **Dashboard de saúde do sistema** — página mostrando status de todos os componentes
- ☐ **Alerta quando margem estiver baixa** — notificação automática
- ☐ **Log de erros centralizado** — arquivo único com todos os erros do sistema

Engines

- ☐ **Golden Whale Hunter** — verificar se está operacional ou só em modo alerta
- ☐ **Azure OpenAI** — pasta server/src/azure/ existe mas não parece integrada
- ☐ **Machine Learning** — integrar modelo XGBoost treinado (cerebro_smc_btcsd.pkl) com os engines
- ☐ **Otimizar Alpha Robot** — 166 trades no histórico mas só 3 recentes; verificar se está operando ativamente
- ☐ **Otimizar Supreme Engine** — 299 trades mas 0 no MT5 history; verificar sync
- ☐ **Pipeline Python-TypeScript** — integrar robo_execucao.py com o servidor Node

Usabilidade

- ☐ **Notificação sonora** — SoundService.js existe mas pode não estar conectado aos eventos
- ☐ **Modo escuro/claro** — alternância entre temas
- ☐ **Exportar relatórios em PDF** — jspdf já está nas dependências
- ☐ **Atalhos de teclado** — navegação rápida entre telas
- ☐ **Pesquisa global** — buscar qualquer coisa (símbolo, robô, trade)